

String Coding Problems

Five problems that come up constantly. Each one has an easy answer and a better answer, and the interview is usually about knowing both and saying why.

1. Reverse a String

Turn "hello" into "olleh".

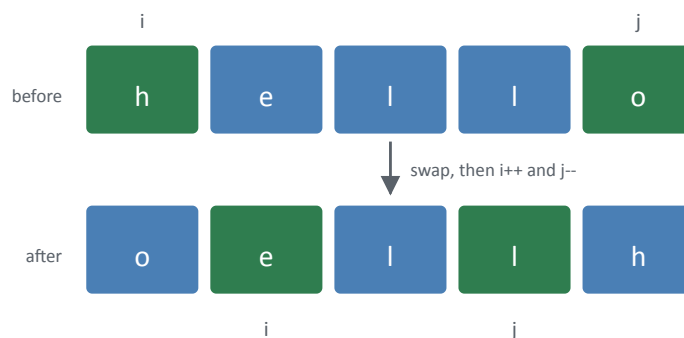
The one-line answer is the right first thing to say:

```
public class Main {  
    public static void main(String[] args) {  
        String s = "hello";  
        System.out.println(new StringBuilder(s).reverse().toString());  
    }  
}
```

Output:

olleh

The answer they usually want is the two-pointer swap, because it shows the mechanics rather than the library.



The pointers move toward each other and stop when they meet. A string of odd length leaves its middle character alone, which is correct.

```
static String reverse(String s) {
    char[] c = s.toCharArray();
    int i = 0, j = c.length - 1;
    while (i < j) {
        char tmp = c[i];
        c[i++] = c[j];
        c[j--] = tmp;
    }
    return new String(c);
}
```

The follow-up worth knowing is what happens to an emoji.

```
String s = "ab😊";           // a, b, and one emoji
String library = new StringBuilder(s).reverse().toString();
String manual = reverse(s);

System.out.println(library);
System.out.println(library.equals(manual));
System.out.println(s.codePointCount(0, s.length()));
System.out.println(manual.codePointCount(0, manual.length()));
```

Output:

```
😊ba
false
3
4
```

`StringBuilder.reverse()` keeps surrogate pairs together. The manual swap reverses the two halves of the emoji, so three characters become four broken ones. Saying that out loud is usually worth more than the loop itself.

Approach	Time	Space
<code>StringBuilder.reverse()</code>	$O(n)$	$O(n)$
Two pointers on a <code>char[]</code>	$O(n)$	$O(n)$ for the array
Recursion	$O(n)$	$O(n)$ stack, and it overflows on long input

2. Reverse words in a String

Turn "the sky is blue" into "blue is sky the".

Ask which reversal is meant first. There are two readings, and answering the wrong one wastes the question:

"the sky is blue" → "blue is sky the" the word order
"the sky is blue" → "eht yks si eulb" each word in place

The first is what is nearly always meant.

```
public class Main {  
    public static void main(String[] args) {  
        String s = "  the  sky is blue  ";  
        String[] words = s.trim().split("\\s+");  
  
        StringBuilder out = new StringBuilder();  
        for (int i = words.length - 1; i >= 0; i--) {  
            out.append(words[i]);  
            if (i > 0) out.append(' ');  
        }  
        System.out.println("[ " + out + " ]");  
    }  
}
```

Output:

[blue is sky the]

The input is where the marks are. The example deliberately has leading, trailing and repeated spaces, because that is what the interviewer adds after the first working answer.

Detail	Why it matters
<code>trim()</code> before <code>split</code>	<code>" a b".split("\\s+")</code> returns an empty first element
<code>\\s+</code> not <code>" "</code>	Splitting on one space turns <code>"a b"</code> into three elements
Building with <code>StringBuilder</code>	Joining in a loop with <code>+</code> is $O(n^2)$

The same thing with the library, once the manual version is on the board:

```
String[] words = s.trim().split("\\s+");  
Collections.reverse(Arrays.asList(words));  
System.out.println(String.join(" ", words));
```

For the second reading, each word is reversed where it stands:

```
String flipped = Arrays.stream(s.trim().split("\\s+"))
    .map(w -> new StringBuilder(w).reverse().toString())
    .collect(Collectors.joining(" "));
System.out.println(flipped);
```

Output:

```
eht yks si eulb
```

3. Check palindrome

A palindrome reads the same forward and backward.

The two-pointer version is the one to write, because it uses no extra memory and stops early on the first mismatch.

```
static boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        if (s.charAt(i) != s.charAt(j)) return false;
        i++;
        j--;
    }
    return true;
}
```

```
System.out.println(isPalindrome("racecar"));
System.out.println(isPalindrome("hello"));
System.out.println(isPalindrome(""));
System.out.println(isPalindrome("a"));
```

Output:

```
true
false
true
true
```

An empty string and a single character are both palindromes, and the loop gives that answer without a special case.

The real question is usually the messy version: ignore case, spaces and punctuation.

```
static boolean isCleanPalindrome(String s) {
    String clean = s.replaceAll("[^A-Za-z0-9]", "").toLowerCase();
    return isPalindrome(clean);
}
```

```
System.out.println(isCleanPalindrome("A man, a plan, a canal: Panama"));
System.out.println(isCleanPalindrome("race a car"));
```

Output:

```
true
false
```

Cleaning first costs a second pass and a new string. The better answer skips the copy and advances the pointers past anything that is not a letter or digit, comparing in place. Mentioning that is usually enough without writing it.

The reverse-and-compare version is fine to name as the one-liner:

```
return s.equals(new StringBuilder(s).reverse().toString());
```

It reads well and does the most work: it always builds a whole new string, and it never stops early.

A palindrome number is the same question without a string. Converting the number to text is the obvious answer, and the expected one reverses half the digits instead:

```
static boolean isPalindrome(int x) {
    if (x < 0 || (x % 10 == 0 && x != 0)) return false;
    int rev = 0;
    while (x > rev) {
        rev = rev * 10 + x % 10;
        x /= 10;
    }
    return x == rev || x == rev / 10;
}
```

```
System.out.println(isPalindrome(121));
System.out.println(isPalindrome(1221));
System.out.println(isPalindrome(-121));
System.out.println(isPalindrome(10));
```

Output:

```
true
true
false
false
```

Two guards carry the edge cases. A negative number is never a palindrome because of the sign, and any number ending in zero would need a leading zero to match. Stopping when `x > rev` means only half the digits are ever reversed, so the result cannot overflow.

The harder follow-up is the longest palindromic substring. The approach to name is expand-around-center: treat every position, and every gap between two positions, as a possible middle and grow outward while the characters match. That is $O(n^2)$ time and $O(1)$ space, and it is the answer expected unless somebody asks for Manacher's algorithm by name.

4. Count characters

This question is ambiguous, and the ambiguity is the point. It means one of three things.

The length of the string:

```
System.out.println("programming".length());
```

Output:

```
11
```

The number of times one character appears:

```
String s = "programming";

long gs = s.chars().filter(c -> c == 'g').count();
System.out.println(gs);

int count = 0;
for (char c : s.toCharArray()) {
    if (c == 'g') count++;
}
System.out.println(count);
```

Output:

2
2

The number of real characters, which is not always the length:

```
String emoji = "ab😄";

System.out.println(emoji.length());
System.out.println(emoji.codePointCount(0, emoji.length()));
```

Output:

4
3

length() counts UTF-16 code units, not characters. Anything outside the Basic Multilingual Plane, which includes every emoji, takes two of them. For English text the two numbers always agree, which is why this is only discovered in production.

Question	Method
How long is it?	<code>length()</code>
How many characters?	<code>codePointCount(0, length())</code>
How many of this one?	<code>chars().filter(...).count()</code>
How many of each?	Section 5

5. Count frequency of characters

Count how many times each character appears.

```
public class Main {
    public static void main(String[] args) {
        String s = "programming";
        Map<Character, Integer> freq = new TreeMap<>();

        for (char c : s.toCharArray()) {
            freq.merge(c, 1, Integer::sum);
        }
        System.out.println(freq);
    }
}
```

Output:

```
{a=1, g=2, i=1, m=2, n=1, o=1, p=1, r=2}
```

merge is the idiom worth remembering. It replaces the older `put(c, map.getOrDefault(c, 0) + 1)` and reads as what it does: insert 1, or add 1 to what is there.

A **TreeMap** was chosen so the output has a defined order. A **HashMap** gives the same counts in an order nobody should rely on, and a **LinkedHashMap** gives first-seen order.

When the alphabet is known and small, use an array. No boxing, no hashing:

```
int[] counts = new int[26];
for (char c : s.toCharArray()) {
    counts[c - 'a']++;
}
System.out.println(counts['g' - 'a']);
```

Output:

```
2
```

That only holds for lower-case ASCII letters. `new int[128]` indexed by the character itself covers all of ASCII without the assumption.

The stream version is shorter and slower, and is worth knowing because it gets asked for by name:

```
Map<Character, Long> freq = s.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, TreeMap::new, Collectors.counting()));
```

Counting is the base for the next three problems, and each of them is this loop followed by one more pass.

6. Find the first non-repeating character

In `"swiss"`, `s` appears three times and `w` is the first character that appears once.

Two passes are unavoidable. Nothing can be decided about the first character until the whole string has been read, so the first pass counts and the second finds. Anyone offering a one-pass answer has misread the problem.

```
static Character firstUnique(String s) {
    Map<Character, Integer> freq = new LinkedHashMap<>();
    for (char c : s.toCharArray()) {
        freq.merge(c, 1, Integer::sum);
    }
    for (Map.Entry<Character, Integer> e : freq.entrySet()) {
        if (e.getValue() == 1) return e.getKey();
    }
    return null;
}
```

```
System.out.println(firstUnique("swiss"));
System.out.println(firstUnique("aabb"));
```

Output:

```
w
null
```

LinkedHashMap, not **HashMap**. The second pass walks the map rather than the string, so it only finds the *first* unique character if the map keeps insertion order. This is the single most common bug in this problem.

The array version avoids that trap entirely, because the second pass walks the string:

```
static int firstUniqueIndex(String s) {
    int[] counts = new int[256];
    for (char c : s.toCharArray()) counts[c]++;
    for (int i = 0; i < s.length(); i++) {
        if (counts[s.charAt(i)] == 1) return i;
    }
    return -1;
}
```

```
System.out.println(firstUniqueIndex("swiss"));
System.out.println(firstUniqueIndex("aabb"));
```

Output:

```
1
-1
```

Returning an index with **-1** for "none" is the usual signature, and it sidesteps the question of what character to return when there is no answer.

7. Find duplicate characters in a String

Report every character that appears more than once.

The one-pass version uses two sets, and reads as exactly what it does:

```
public class Main {
    public static void main(String[] args) {
        String s = "programming";
        Set<Character> seen = new HashSet<>();
        Set<Character> dupes = new LinkedHashSet<>();

        for (char c : s.toCharArray()) {
            if (!seen.add(c)) dupes.add(c);
        }
        System.out.println(dupes);
    }
}
```

Output:

```
[r, m, g]
```

Set.add returns false when the element was already there. That one fact removes the need to count anything, and **dupes** is a set so a character repeated three times is still reported once.

When the counts are wanted too, go back to the frequency map and drop the singles:

```
Map<Character, Integer> freq = new TreeMap<>();
for (char c : s.toCharArray()) {
    freq.merge(c, 1, Integer::sum);
}
freq.entrySet().removeIf(e -> e.getValue() == 1);
System.out.println(freq);
```

Output:

```
{g=2, m=2, r=2}
```

Ask whether case matters. In "Programming" the P and p are different characters, and whether that is a duplicate depends on a rule nobody has stated yet.

8. Check anagram

Two strings are anagrams when one is a rearrangement of the other: "listen" and "silent".

Sorting is the answer to give first, because it is obviously correct:

```
static boolean isAnagramBySorting(String a, String b) {  
    if (a.length() != b.length()) return false;  
    char[] x = a.toCharArray();  
    char[] y = b.toCharArray();  
    Arrays.sort(x);  
    Arrays.sort(y);  
    return Arrays.equals(x, y);  
}
```

Counting is the answer to write, because it is $O(n)$ rather than $O(n \log n)$. One array, incremented for the first string and decremented for the second:

```
static boolean isAnagram(String a, String b) {  
    if (a.length() != b.length()) return false;  
    int[] counts = new int[26];  
    for (int i = 0; i < a.length(); i++) {  
        counts[a.charAt(i) - 'a']++;  
        counts[b.charAt(i) - 'a']--;  
    }  
    for (int c : counts) {  
        if (c != 0) return false;  
    }  
    return true;  
}
```

```
System.out.println(isAnagram("listen", "silent"));  
System.out.println(isAnagram("hello", "world"));  
System.out.println(isAnagram("ab", "abc"));
```

Output:

```
true  
false  
false
```

The length check first is not an optimization. Without it, `counts` would be read past the end of the shorter string, so it is what makes the single loop valid at all.

Detail	Decision
Different lengths	Never anagrams. Return early
Case	"Listen" and "silent" only match after lower-casing
Spaces and punctuation	Strip them first, if phrases are allowed
Beyond lower-case ASCII	<code>new int[26]</code> breaks. Use a map

The follow-up is **grouping anagrams**, and the sorted string becomes the key:

```
String[] words = {"eat", "tea", "tan", "ate", "nat", "bat"};
Map<String, List<String>> groups = new TreeMap<>();

for (String w : words) {
    char[] c = w.toCharArray();
    Arrays.sort(c);
    groups.computeIfAbsent(new String(c), k -> new ArrayList<>()).add(w);
}
System.out.println(groups);
```

Output:

```
{abt=[bat], aet=[eat, tea, ate], ant=[tan, nat]}
```

`computeIfAbsent` is the idiom here, exactly as `merge` was for counting.

The pieces together

Problem	Best approach	Time	Space
Reverse a string	Two pointers on <code>char[]</code>	$O(n)$	$O(n)$
Reverse words	<code>trim</code> , <code>split("\\s+")</code> , build backward	$O(n)$	$O(n)$
Palindrome	Two pointers, no copy	$O(n)$	$O(1)$
Palindrome number	Reverse half the digits	$O(\log n)$	$O(1)$
Count one character	Single pass	$O(n)$	$O(1)$
Count all characters	Map, or an array for a known alphabet	$O(n)$	$O(k)$
First non-repeating	Count, then walk the string again	$O(n)$	$O(k)$
Duplicate characters	One pass, a seen set and a duplicate set	$O(n)$	$O(k)$
Anagram	One <code>int[26]</code> , up for one string and down for the other	$O(n)$	$O(1)$

Problem	Best approach	Time	Space
Group anagrams	Sorted string as the map key	$O(n \cdot m \log m)$	$O(n \cdot m)$

Three idioms cover most of the code above, and they are worth knowing by name: `map.merge` for counting, `map.computeIfAbsent` for grouping, and the fact that `set.add` returns false when the element was already present.

Four habits cover most of what is being marked:

Say the library answer, then write the manual one. Knowing `StringBuilder.reverse()` exists is not a weakness, and pretending otherwise wastes time.

Ask about the input before writing. Repeated spaces, empty strings, mixed case and non-ASCII characters are where every follow-up comes from, and asking first is faster than being corrected.

Build strings with `StringBuilder` in any loop. Concatenating with `+` inside a loop is $O(n^2)$, and it is the most common thing marked down in these answers.

Say the complexity without being asked. One sentence at the end of each answer, covering time and space, closes the question properly.